

Kiosk Activation — Complete Architecture, Schema & Live-Debug Guide

Purpose: give the developer a complete, self-contained picture of how the Hall Pass door **kiosk** gets activated, the **two** entry points that exist, exactly what they share, and — most importantly — **why the self-serve teacher flow works in development but not on live** (while the admin cards work on both).

Read this first (TL;DR): There are **two entry points** for handing a kiosk its activation code, but only **one** underlying credential + activation system. They converge on the same database tables and the same activation route. The admin **card** flow works on live. The **self-serve teacher** flow does not. The two flows differ **only in the front half** (how the code is generated and presented). The single most likely cause is that the **/kiosk-code** page is **not being served on the live host**, and/or the client build's **BASE_URL** is wrong on live. The admin card **never touches /kiosk-code** and embeds a **server-supplied** URL — which is precisely why cards work and self-serve doesn't. See §7.

1. The two entry points at a glance

	(A) Admin "Kiosk Cards"	(B) Teacher self-serve
Who starts it	Admin (single or bulk)	The teacher, on their own phone
Where in UI	Admin Kiosk Cards panel (<code>KioskCardsPanel.tsx</code>)	Hall Pass gear → "Get kiosk URL" (<code>TeacherDestinationPicker.tsx</code>)
Code generator (server)	<code>POST /api/kiosk/enroll-tokens/regenerate/:staffId</code> , <code>POST /api/kiosk/enroll-tokens/bulk-generate</code>	<code>POST /api/kiosk/my-code/regenerate</code>
Underlying issuer	<code>issueEnrollToken()</code>	<code>issueEnrollToken()</code> — same function
How the code reaches the kiosk	Printed PDF (QR + Code 128 + 6-digit PIN), built server-side by <code>kioskCardsPdf.ts</code>	Phone mirror page <code>/kiosk-code#t=<token>&p=<pin></code> (<code>KioskCodeMirror.tsx</code>) renders the QR on screen
URL base used for the QR	Server-supplied <code>baseUrl</code> param (<code>kioskCardsPdf.ts</code>)	Client <code>window.location.origin + import.meta.env.BASE_URL</code>
Kiosk activation route	<code>POST /api/kiosk/activate-by-enrollment</code> (shared)	<code>POST /api/kiosk/activate-by-enrollment</code> (shared)
Writes to	<code>kiosk_enroll_tokens</code> (credential) + <code>kiosk_activations</code> (session)	same two tables

Key takeaway: (A) and (B) are two faucets on the same pipe. They are NOT two independent kiosk systems. The only real difference is **how the QR/PIN is presented** (printed vs. on-screen) and **who is allowed to trigger the rotate**.

2. Data model (source of truth: `lib/db/src/schema/`)

2.1 `kiosk_enroll_tokens` — the per-teacher credential

File: `lib/db/src/schema/kioskEnrollTokens.ts`

This is the long-lived credential a teacher "carries" (as a printed card OR as an on-screen code). **At most one live token per teacher per school.**

Column	Type	Notes
<code>id</code>	serial PK	
<code>school_id</code>	integer NOT NULL	tenant
<code>staff_id</code>	integer NOT NULL	the teacher this code belongs to
<code>token_hash</code>	text NOT NULL UNIQUE	sha256 of the raw 256-bit token
<code>pin_hash</code>	text (nullable)	bcrypt of the 6-digit PIN
<code>pin_encrypted</code>	text (nullable)	AES-256-GCM copy (purpose <code>kiosk-pin-v1</code>) so the OWNER can re-reveal it
<code>label</code>	text (nullable)	
<code>created_at</code>	timestampz NOT NULL default now	
<code>created_by_staff_id</code>	integer (nullable)	
<code>rotated_at</code>	timestampz (nullable)	
<code>revoked_at</code>	timestampz (nullable)	non-null = dead
<code>revoked_by_staff_id</code>	integer (nullable)	
<code>last_used_at</code>	timestampz (nullable)	stamped on successful activation

Critical index (interdependence enforcer):

```
kiosk_enroll_tokens_one_live_per_staff
  UNIQUE (school_id, staff_id) WHERE revoked_at IS NULL
```

→ There can be only **one** un-revoked token per teacher. Issuing a new one (from **either** entry point) must **revoke the old one in the same transaction**. **If this partial unique index is missing on live, both flows can silently corrupt** (two live tokens, or the revoke-then-insert throwing 23505).

2.2 kiosk_activations — the live device session

File: `lib/db/src/schema/kioskActivations.ts`

Created when a code is accepted at the kiosk. One live session per room.

Column	Type	Notes
<code>id</code>	serial PK	
<code>school_id</code>	integer NOT NULL	stamped from the teacher (activate route is unauthenticated)
<code>token_hash</code>	text NOT NULL UNIQUE	
<code>room</code>	text NOT NULL	the classroom
<code>staff_id</code>	integer NOT NULL	teacher the kiosk is FOR (masthead name)
<code>activated_at</code>	timestampz NOT NULL default now	
<code>expires_at</code>	timestampz NOT NULL	TTL (<code>ENROLL_TTL_MS</code>)
<code>device_label</code> / <code>device_fingerprint</code>	text (nullable)	
<code>deactivated_at</code> / <code>deactivated_by_staff_id</code>		non-null = ended
<code>enroll_token_id</code>	integer (nullable)	which <code>kiosk_enroll_tokens</code> row was used (NULL = legacy email+password path)
<code>activated_by_staff_id</code>	integer (nullable)	who triggered it (differs from <code>staff_id</code> for sub/proxy)
<code>proxy_for_staff_id</code>	integer (nullable)	sub coverage
<code>session_kind</code>	text (nullable)	<code>password</code> <code>enroll</code> <code>proxy</code>
<code>on_time_ended_key</code>	text (nullable)	On-Time Attendance "Done" marker

Critical index:

```
kiosk_activations_one_live_per_room
  UNIQUE (school_id, room) WHERE deactivated_at IS NULL
```

→ Race-safe guard: only one live kiosk per room. A concurrent second activate fails with Postgres 23505 (handled in the activate route).

2.3 Related (for reference, not required to fix this issue)

- `lib/db/src/schema/kioskViewerTokens.ts` — read-only "kiosk viewer" (`/kiosk-view`).
- `lib/db/src/schema/hallPassQueue.ts` — the pass queue the kiosk drives.

Migration note: these tables/indexes must all exist on the **live** DB. Additive columns in this project are applied at server boot via `seed.ts` ensure-schema helpers; confirm they actually ran against production.

3. The shared core (both flows converge here)

File: `artifacts/api-server/src/routes/kiosk.ts`

3.1 `issueEnrollToken()` — the single code minter

Used by **both** entry points (reasons: `regenerate`, `bulk_generate`, `card_print`, `self_regenerate`). It: 1. Generates a 256-bit raw token + a 6-digit PIN. 2. Stores `token_hash` (sha256), `pin_hash` (bcrypt), `pin_encrypted` (AES-GCM). 3. **In one transaction: revokes any existing live token for (`school_id`, `staff_id`), then inserts the new one** (keeps the partial unique index happy). 4. Writes an audit row to `admin_notifications`.

3.2 `activateForTeacher()` + the activation routes — the single door

- `POST /api/kiosk/activate-by-enrollment` — body `{ enrollToken, room, ... }`. Looks up the token by hash, verifies the teacher is active, then activates.
- `POST /api/kiosk/activate-by-pin` — body `{ pin, room, schoolId }`. Same, but matches by PIN (bcrypt), IP-throttled because it's unauthenticated + expensive.

Both write the `kiosk_activations` row with `session_kind: "enroll"`. **Both flows (A and B) use this exact same door.** Since admin cards work on live, **this door is proven working on live.**

4. End-to-end: flow (A) Admin card

1. Admin opens Kiosk Cards panel → `regenerate` / `bulk-generate`.
2. Server `issueEnrollToken()` mints the token; `kioskCardsPdf.ts` renders a PDF whose QR encodes `${baseUrl}?enroll=${enrollToken}` where `baseUrl` is **passed in by the server** (e.g. `https://<school>.pulseedu. .... /kiosk`).
3. Teacher holds the printed QR (or types the PIN) at the kiosk.
4. Kiosk (`Kiosk.tsx`) reads `?enroll=<token>` from its own URL → `POST /api/kiosk/activate-by-enrollment` → `kiosk_activations` row. □

Note flow (A) never loads `/kiosk-code` and never uses the client's `import.meta.env.BASE_URL`. This is the whole reason it's immune to the live bug.

5. End-to-end: flow (B) Teacher self-serve

1. Teacher opens Hall Pass gear → "Get kiosk URL" (`TeacherDestinationPicker.tsx`).
2. Client calls `POST /api/kiosk/my-code/regenerate` → gets `{ enrollToken, pin }`.

3. Client builds a **mirror URL** and shows it as a QR: `${window.location.origin}${import.meta.env.BASE_URL}kiosk-code#t=<token>&p=<pin>` (The token/PIN live in the URL **hash**, so they never hit the server/logs.)
4. Teacher scans that with their phone → phone opens `/kiosk-code` → `KioskCodeMirror.tsx` renders the real activation QR encoding `${window.location.origin}${import.meta.env.BASE_URL}kiosk?enroll=<token>`.
5. Teacher holds the phone up to the kiosk → same `?enroll=` door as (A). ☐ (in dev)

Flow (B) has two extra client-only dependencies that (A) does not: - The `/kiosk-code` route **must be served** by the SPA on the live host. - `import.meta.env.BASE_URL` (baked at build time) must match the live path.

6. Client routing / path dispatch (why order matters)

File: `artifacts/client/src/main.tsx` — a single Vite bundle dispatches by path:

```
/kiosk-view/<token> → KioskViewer      (checked FIRST)
/kiosk-code#...     → KioskCodeMirror  (checked BEFORE /kiosk)
/kiosk              → Kiosk            (the real activation screen)
```

`isKioskCode = path.includes("/kiosk-code")` is evaluated **before**
`isKiosk = ... path.includes("/kiosk")`, so `/kiosk-code` must not be swallowed by `/kiosk`. **If the live host doesn't serve the SPA at `/kiosk-code` (SPA fallback / proxy path), the phone gets a 404 / blank and the whole self-serve flow dies at step 4** — even though the underlying activation works.

7. WHY IT WORKS IN DEV BUT NOT LIVE — prioritized checklist

The break is in flow (B)'s **front half** (steps 2–4 above), because the shared door (proven by working cards) is fine. Check in this order:

- ① **`/kiosk-code` is not served on the live host (MOST LIKELY).** - Test: on live, open `https://<live-domain>/<base>/kiosk-code#t=abcdef1234567890&p=123456` directly in a browser. - Dev shows the mirror page with a QR. If live 404s / blanks / redirects, this is it. - Cause: SPA fallback / reverse-proxy path routing doesn't route `/kiosk-code` to the client bundle (the shared proxy matches by path). Cards bypass this page entirely, which is why they work.
- ② **Client `BASE_URL` is wrong in the live build.** - The self-serve QRs are built from `import.meta.env.BASE_URL`. If the live build was produced with a different base path than the domain actually serves, the generated `/kiosk-code` and the inner `/kiosk?enroll=` URLs point to a path that doesn't exist on live. - The **card PDF** uses a **server-supplied `baseUrl`** instead, so it's immune — confirm the server's card `baseUrl` and the client's `BASE_URL` agree. - Test: generate a self-serve code on live, read the QR (any QR reader), and verify the URL it encodes is a real, reachable live URL.
- ③ **`POST /api/kiosk/my-code/regenerate` failing on live.** - Requires an authenticated staff session (`requireStaff`). If the staff auth cookie/session isn't valid in that context on live, step 2 fails before a code is ever shown. - Test: watch the network call when the teacher taps "Get kiosk URL" on live.

④ **"Reveal my existing PIN" (GET /api/kiosk/my-pin) failing.** - This decrypts `pin_encrypted` with the app's secret key. **If the encryption master key / secret env var isn't set (or differs) on live, decrypt throws.** Note: `my-code/regenerate` returns a fresh raw PIN directly and does NOT need decrypt, so generating a new code can work while revealing an old one fails. - Confirm the secret-crypto env var(s) exist on live.

⑤ **DB schema/index drift on live.** - Confirm `kiosk_enroll_tokens` + `kiosk_activations` exist with BOTH partial unique indexes (\$2.1, \$2.2). A missing `one_live_per_staff` index makes the revoke-then-insert in `issueEnrollToken()` behave differently under load.

8. Morning fix checklist (do in order)

1. **Open the live URL** `.../kiosk-code#t=<16+chars>&p=123456` directly.
2. Blank/404 → fix SPA routing / proxy for `/kiosk-code` (**most likely fix**).
3. Renders a QR → routing is fine; go to step 2.
4. **Scan a live self-serve QR** and read the raw URL it encodes. Confirm both the mirror URL and the inner `?enroll=` URL are reachable live paths. If not → fix the client build `BASE_URL` (and confirm it matches the server card `baseUrl`).
5. **Watch the network** for `POST /api/kiosk/my-code/regenerate` on live — must return 201 with `{ enrollToken, pin }`. 401/500 → auth/secret issue.
6. **Confirm secret-crypto env** is set on live (needed for `GET /kiosk/my-pin`).
7. **Confirm DB** has both tables + both partial unique indexes on live.
8. Re-test the full self-serve path end to end on a real device.

9. Interdependence — READ BEFORE removing anything

- **One live token per teacher.** Generating a card (A) **revokes** that teacher's self-serve code (B), and regenerating self-serve (B) revokes the card (A). They rotate the **same** `kiosk_enroll_tokens` slot.
- **Field implication:** if an admin bulk-prints cards after a teacher already self-activated, the teacher's on-screen code stops working (it was revoked). This can look like "self-serve randomly breaks."
- **If you later remove the admin card flow:** you may delete `KioskCardsPanel`, `kioskCardsPdf.ts`, and the admin `enroll-tokens/*` routes **without breaking self-serve** — but you must KEEP `kiosk_enroll_tokens`, `issueEnrollToken()`, `activate-by-enrollment` / `activate-by-pin`, `kiosk_activations`, the `/kiosk-code` route, and `my-code/regenerate`. Those are the self-serve backbone. Do NOT "scrap the kiosk" — only the card presentation layer is removable.

10. File / route index

Server (`artifacts/api-server/src/`) - `routes/kiosk.ts` — all kiosk routes + `issueEnrollToken()` + `activateForTeacher()` . - Self-serve: `POST /kiosk/my-code/regenerate`, `GET /kiosk/my-pin` . - Admin cards: `POST /kiosk/enroll-tokens/regenerate/:staffId`, `POST /kiosk/enroll-tokens/bulk-generate`, `GET /kiosk/enroll-tokens`, `POST /kiosk/cards.pdf`, `POST /kiosk/teacher-badges.pdf` . - Shared door: `POST /kiosk/`

`activate-by-enrollment`, `POST /kiosk/activate-by-pin`. - `lib/kioskCardsPdf.ts` — printable card PDF (uses server-supplied `baseUrl`).

Client (`artifacts/client/src/`) - `main.tsx` — path dispatch (`/kiosk-view`, `/kiosk-code`, `/kiosk`). - `Kiosk.tsx` — the activation screen; reads `?enroll=` and calls the door. - `KioskCodeMirror.tsx` — the `/kiosk-code` phone mirror page (self-serve only). - `components/TeacherDestinationPicker.tsx` — "Get kiosk URL": calls `my-code/regenerate` and builds the `/kiosk-code#t=...&p=...` mirror URL. - `components/KioskCardsPanel.tsx` — admin card management UI.

Schema (`lib/db/src/schema/`) - `kioskEnrollTokens.ts`, `kioskActivations.ts` (+ `kioskViewerTokens.ts`).